

# Applied Programming & Design Principles

## S - 01

Ms. ALF. Sajeetha

Lecturer - IT

# LO1 - Investigate the impact of SOLID development principles on the OOP paradigm

## Object-orientated programming (OOP) paradigm characteristics

- Understand the OO characteristics and their application in developing code,
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Constructors and Destructors
- Sub-objects, Interface
- Method overriding and redefinition
- Templates and Containers

# Object-orientated programming (OOP) paradigm characteristics

- <https://medium.com/pythoneers/mastering-the-art-of-object-oriented-programming-in-python-df04c83cc15b>
- <https://medium.com/swlh/the-4-pillars-of-oop-in-python-9daaca4c0d13>

# Introduction to Object-Oriented Programming

- Grady Booch has defined object-oriented programming as *“a method of implementation in which programs are organized as cooperative collections of objects, each of which represents an instance of some class, and whose classes are all members of a hierarchy of classes united via inheritance relationships”*.
- The main concept of OOPs is to bind the data and the functions that work on that together as a single unit so that no other part of the code can access this data.

"Object-oriented programming (OOP) is a programming paradigm based upon objects (having both data and methods) that aims to incorporate the advantages of modularity and reusability"

"Objects, which are usually instances of classes, are used to interact with one another to design applications and computer programs"

# The important features of object oriented programming

- Bottom–up approach in program design
- Programs organized around objects, grouped in classes
- Focus on data with methods to operate upon object's data
- Interaction between objects through functions
- Reusability of design through creation of new classes by adding features to existing classes

Some examples of object-oriented programming languages are C++, Java, Smalltalk, Delphi, C#, Perl, Python, Ruby, and PHP.

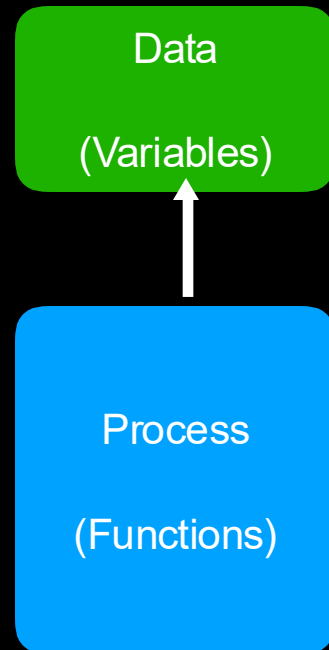
# An object-oriented language must provide support for three key language features

- Abstract data types - abstraction and encapsulation
- Inheritance - Parent and Sub-classes
- Polymorphism - Dynamic binding of method call to methods.

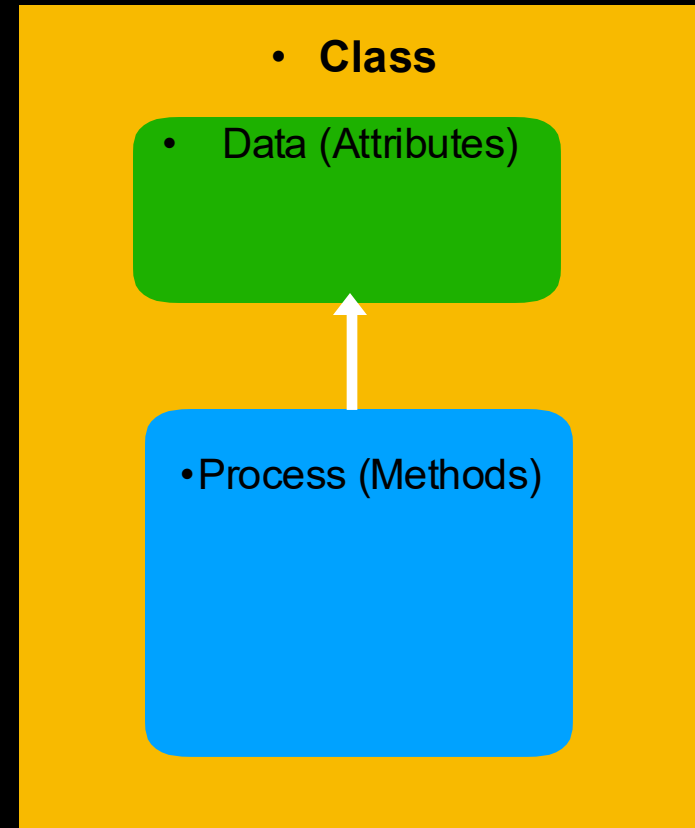
# **Abstract Data Types(ADT)**

## **Abstraction & Encapsulation**

# Abstraction and Encapsulation



**Traditional Procedural Programming**



**Object-Oriented Programming**

# Classes, Objects, Methods and Variables

# Classes, Objects, Methods and Messages

- The abstract data types in object-oriented languages are usually called **classes**.
- As with instances of abstract data types, class instances are called **objects**.
- A class that is defined through inheritance from another class is a **derived class**, a **subclass**, or a **child class**.
- A class from which the new class is derived is its **base class**, **superclass**, or **parent class**.
- The **subprograms** that define the operations on objects of a class are called **methods**.
- The calls to methods are sometimes called **messages**.
- The entire collection of methods of a class is called the **message protocol**, or **message interface**, of the class.
- **Computations** in an object-oriented program are specified by messages sent from objects to other objects, or in some cases, to classes.

# Class

- A class is a collection of objects. A class contains the blueprints or the prototype from which the objects are being created. It is a logical entity that contains some attributes and methods.
- To understand the need for creating a class let's consider an example, let's say you wanted to track the number of dogs that may have different attributes like breed, and age. If a list is used, the first element could be the dog's breed while the second element could represent its age. Let's suppose there are 100 different dogs, then how would you know which element is supposed to be which? What if you wanted to add other properties to these dogs? This lacks organization and it's the exact need for classes.

- Classes are created by keyword class.
- Attributes are the variables that belong to a class.
- Attributes are always public and can be accessed using the dot (.) operator. Eg.: Myclass.Myattribute

Class Definition Syntax:

```
class ClassName:  
    # Statement-1  
    .  
    .  
    .  
    # Statement-N
```

## Creating an Empty Class in Python

In the above example, we have created a class named Dog using the class keyword.

Python



```
# Python3 program to  
# demonstrate defining  
# a class
```



```
class Dog:  
    pass
```



# Variables

- Class variables belong to the class, rather than its object, so there is only one copy for the class. (Class Attributes)
- Every object of a class has its own set of instance variables, which store the object's state. (Instance Attributes)
- The only difference between two objects of the same class is the state of their instance variables.

# Cohesion and Coupling

- **Cohesion** refers to how closely related and focused the responsibilities of a single module or component are. A highly cohesive module performs a single task or a group of related tasks. High cohesion within a module is desirable as it makes the module easier to understand, maintain, and reuse.
- **Coupling** refers to the degree of interdependence between different modules or components. Low coupling means that changes in one module have minimal impact on others, making the system easier to maintain and extend. High coupling, on the other hand, means that modules are heavily dependent on each other, which can lead to a fragile system.

# Subprograms and Methods

- Methods are similar to subprograms. Both are collections of code that perform some computation.
- Both can take parameters and return results.
- Passing a message is different from calling a subprogram. A subprogram typically processes data that is either passed to it by its caller as a parameter or is accessed non-locally or globally.
- A message that is sent to an object is a request to execute one of its methods. At least some of the data on which the method is to operate is part of the object itself.
- Objects have methods that define processes the object can perform on itself.
- Because the objects are of abstract data types, these should be the only ways to manipulate the data of the object.

# Methods

- Classes have two kinds of methods: instance and class methods.
- Class methods can perform operations on the class, and possibly also on the objects of the class. They can be called by prefixing their names with either the class name or a variable that references one of their instances.
- If a class defines a class method, that method can be called even if there are no instances of the class. A class method could be used to create an instance of the class.
- The new method is said to override the inherited method, which is then called an overridden method.
- The purpose of an overriding method is to provide an operation in the subclass that is similar to one in the parent class but is customised for objects of the subclass.

- All objects are allocated from the heap and accessed through reference variables.
- Methods in Java must be defined completely in a class.
- A method body must appear with its corresponding method header.
- A Java abstract data type is both declared and defined in a single syntactic unit.
- Definitions are hidden from clients by declaring them to be private.
- Java uses implicit garbage collection of all objects. This allows the programmer to ignore the issue of deallocation of objects and the clutter of deallocation code in the implementations of abstract data types.

- Rather than having **private** and **public** clauses in its class definitions, in Java **access modifiers** can be attached to **method and variable definitions**.
- If an instance variable or method does **not have an access modifier**, it has **package** access.

# Objects

- An object consists of one or more data values that define the object. An object may be, a graphic, a person, a device, a magazine, a book, a tape, a video clip, a sound clip, or some other data value, depending upon the class of which it is a member.
- It is what you, as a programmer, work within the same manner as you work with a specific record in a file.
- Once an object instance has done the work needed, it can be destroyed, which allows for memory to be freed, greatly enhancing the performance of the system.
- **Procedural programs** typically completely load the entire program, which takes up all the resources that the program needs to complete the task; whereas in OOP, objects can be instantiated and destroyed so computer systems can perform much more complex operation with smaller impact on the system resources.

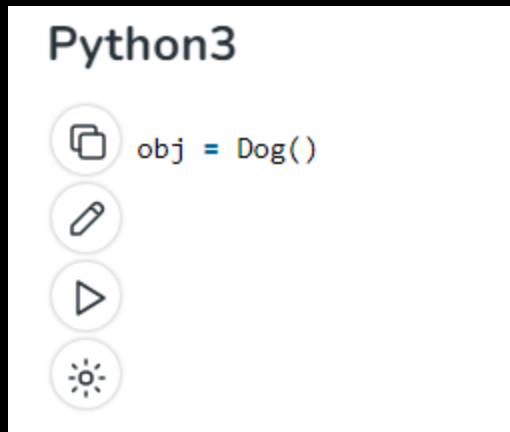
# Python Objects

- The object is an entity that has a state and behavior associated with it. It may be any real-world object like a mouse, keyboard, chair, table, pen, etc. Integers, strings, floating-point numbers, even arrays, and dictionaries, are all objects. More specifically, any single integer or any single string is an object. The number 12 is an object, the string “Hello, world” is an object, a list is an object that can hold other objects, and so on. You’ve been using objects all along and may not even realize it.
- An object consists of:
- **State:** It is represented by the attributes of an object. It also reflects the properties of an object.
- **Behavior:** It is represented by the methods of an object. It also reflects the response of an object to other objects.
- **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

- To understand the state, behavior, and identity let us take the example of the class dog.
- The identity can be considered as the name of the dog.
- State or Attributes can be considered as the breed, age, or color of the dog.
- The behavior can be considered as to whether the dog is eating or sleeping.

# Creating an Object

- This will create an object named obj of the class Dog defined above. Before diving deep into objects and classes let us understand some basic keywords that will we used while working with objects and classes.



# The Python self

- Instance methods must have an extra first parameter in the method definition. We do not give a value for this parameter when we call the method, Python provides it
- If we have a method that takes no arguments, then we still have to have one argument.
- This is similar to this pointer in C++ and this reference in Java.
- When we call a method of this object as `myobject.method(arg1, arg2)`, this is automatically converted by Python into `MyClass.method(myobject, arg1, arg2)` – this is all the special `self` is about.

# **\_\_name\_\_ Attribute**

- `__name__` is a special attribute in Python that gives the name of the module in which the code is running
- For scripts or modules, `__name__` allows you to determine whether the module is being run as the main program or if it's being imported elsewhere

## **Usage of `__name__`**

- As a Script, when a Python script is run directly, `__name__` is set to `"__main__"`
- As a Module, when the same script is imported as a module in another script, `__name__` is set to the name of the module

**`__name__ == "__main__"`**

- The if `__name__ == "__main__"` , block checks whether the script is being run directly or being imported.
- If it's run directly, it will execute the code inside the block.
- If imported, it will not execute that block.

# Pass Statement

- The program's execution is unaffected by the pass statement's inaction. It merely permits the program to skip past that section of the code without doing anything. It is frequently employed when the syntactic constraints of Python demand a valid statement but no useful code must be executed.

# Methods

- In object-oriented programming (OOP), methods are functions that are associated with a class. Within an object, methods are executable sets for performing object specific operation. They define the behavior of objects created from the class.
- Developing the solutions for methods in an object follows exactly the same steps you have been using for procedural programming, since each individual method is a procedural module.
- Each method is cohesive, and data are coupled in the same manner as was done in procedural programming.

# Instance Methods

- Instance methods are the most common type of methods in Python classes.
- They are associated with an instance of the class and have access to the instance itself through the self parameter.
- They can access and modify the instance's attributes.
- They are defined using the def keyword within the class.



```
class Sale:
    def __init__(self, item_name, quantity, price_per_item):
        self.item_name = item_name
        self.quantity = quantity
        self.price_per_item = price_per_item

    def calculate_total(self):
        return self.quantity * self.price_per_item

    def apply_discount(self, discount):
        total = self.calculate_total()
        discounted_total = total * (1 - discount / 100)
        return discounted_total

    def display_sale_info(self):
        print(f"Item: {self.item_name}")
        print(f"Quantity: {self.quantity}")
        print(f"Price per item: ${self.price_per_item:.2f}")
        print(f"Total cost: ${self.calculate_total():.2f}")

# Example usage
sale = Sale("Laptop", 2, 1500.00)

# Display sale information
sale.display_sale_info()

# Calculate the total cost of the sale
total_cost = sale.calculate_total()
print(f"Total cost before discount: ${total_cost:.2f}")

# Apply a 10% discount
discounted_cost = sale.apply_discount(10)
print(f"Total cost after 10% discount: ${discounted_cost:.2f}")
```

# Class Method

- Class methods are associated with the class rather than instances
- They are defined using the `@classmethod` decorator
- Class methods can access class-level data (attributes) and are used to implement behaviors that apply to the class as a whole, not just a single instance
- They take `cls` as the first parameter, which refers to the class itself, rather than an instance of the class. (This is similar to how instance methods take `self` as the first parameter)
- Class methods can be used for operations that involve the class, but not necessarily an instance

```

class Transaction:
    transactions = [] # Class variable to store all transactions

    def __init__(self, transaction_id, amount, description):
        self.transaction_id = transaction_id
        self.amount = amount
        self.description = description
        # Add the instance to the class variable transactions list
        Transaction.transactions.append(self)

    def display_transaction(self):
        return f"Transaction ID: {self.transaction_id}, Amount: ${self.amount}, Description: {self.description}"

    @classmethod
    def get_total_amount(cls):
        total = sum(transaction.amount for transaction in cls.transactions)
        return total

    @classmethod
    def create_transaction(cls, transaction_id, amount, description):
        # Creates a new transaction instance and returns it
        return cls(transaction_id, amount, description)

# Creating instances using the class method
transaction1 = Transaction.create_transaction(1, 150.00, 'Purchase at Store A')
transaction2 = Transaction.create_transaction(2, 200.00, 'Purchase at Store B')

# Accessing the class method to get the total amount of all transactions
total_amount = Transaction.get_total_amount()
print(f"Total Amount of Transactions: ${total_amount}")

# Displaying individual transactions
print(transaction1.display_transaction())
print(transaction2.display_transaction())

```

# Static methods

- Static methods do not have access to the instance (self) or class (cls).
- They cannot modify object state or class state and are completely independent of any instance or class-specific data
- They are defined using the `@staticmethod` decorator.
- They don't have access to the instance or class, and they don't use the self or cls parameters.
- They are similar to regular functions but are defined inside a class for organizational purposes.
- They are often used for utility functions that logically belong to a class but do not require access to the class or instance data.
- For example, a method to validate data, perform calculations, or format strings

```
class Order:
    def __init__(self, order_id, product_name, quantity):
        self.order_id = order_id
        self.product_name = product_name
        self.quantity = quantity

    def display_order(self):
        return f"Order ID: {self.order_id}, Product: {self.product_name}, Quantity: {self.quantity}"

    @staticmethod
    def create_order_summary(order_list):
        summary = []
        for order in order_list:
            summary.append(f"Order ID: {order.order_id}, Product: {order.product_name}, Quantity: {order.quantity}")
        return summary

# Creating instances of the Order class
order1 = Order(1, 'Laptop', 2)
order2 = Order(2, 'Phone', 5)

# Using the static method
orders = [order1, order2]
summary = Order.create_order_summary(orders)
print(summary)
```

# Constructors

- In object-oriented programming (OOP), constructors are special methods used to initialize the attributes of an object when it is created within a class that are automatically invoked when an object is created.
- Constructors are used to initialize the object's attributes or perform any setup necessary for the object to function properly.
- The primary purpose of a constructor is to ensure that the object is in a valid state after it is instantiated.
- In Python, the constructor method is named `__init__`

# Constructors

```
class MyClass:  
    def __init__(self, attribute1, attribute2):  
        self.attribute1 = attribute1  
        self.attribute2 = attribute2
```

---

```
my_object = MyClass(value1, value2)
```

# Types of constructors :

- Default constructor: The default constructor is a simple constructor which doesn't accept any arguments. Its definition has only one argument which is a reference to the instance being constructed.
- Parameterized constructor: constructor with parameters is known as parameterized constructor. The parameterized constructor takes its first argument as a reference to the instance being constructed known as self and the rest of the arguments are provided by the programmer.

# Default constructor

```
class Student:
```

```
# default constructor
```

```
def __init__(self):  
    self.stud = "AMAL"
```

```
# a method for printing data members
```

```
def print_student(self):  
    print(self.stud)
```

```
# creating object of the class
```

```
obj = Student()
```

```
# calling the instance method using the object obj
```

```
obj.print_student()
```

# Parameterized constructor

```
class Addition:
    first = 0
    second = 0
    answer = 0

    # parameterized constructor
    def __init__(self, f, s):
        self.first = f
        self.second = s

    def display(self):
        print("First number = " + str(self.first))
        print("Second number = " + str(self.second))
        print("Addition of two numbers = " + str(self.answer))

    def calculate(self):
        self.answer = self.first + self.second

# creating object of the class
# this will invoke parameterized constructor
obj1 = Addition(1000, 2000)

# creating second object of same class
obj2 = Addition(10, 20)

# perform Addition on obj1
obj1.calculate()

# perform Addition on obj2
obj2.calculate()

# display result of obj1
obj1.display()

# display result of obj2
obj2.display()
```

## Python



```
class MyClass:
    def __init__(self, name=None):
        if name is None:
            print("Default constructor called")
        else:
            self.name = name
            print("Parameterized constructor called with name", self.name)

    def method(self):
        if hasattr(self, 'name'):
            print("Method called with name", self.name)
        else:
            print("Method called without a name")

# Create an object of the class using the default constructor
obj1 = MyClass()

# Call a method of the class
obj1.method()

# Create an object of the class using the parameterized constructor
obj2 = MyClass("John")

# Call a method of the class
obj2.method()
```

# \_\_new\_\_ Method

- `__new__` is a special method in Python that is responsible for creating a new instance of a class. It's called before `__init__` and is rarely overridden, but when it is, it's usually in cases where you need to control the creation of a new instance of a class.
- You might override `__new__` in scenarios where you're working with immutable types like tuples or strings, or if you need to implement a **singleton pattern (ensuring only one instance of a class exists)**.

## How `__new__` Works:

### Creation:

- `__new__` is called with the class itself as its first argument, followed by any additional arguments passed during instance creation.
- Python calls `__new__` when you create a new instance of a class (i.e., when you call `ClassName()`).

### Return:

- `__new__` must return a new instance of the class. If `__new__` does not return an instance of the class (or an instance of a subclass), the `__init__` method will not be called

- `__new__` is responsible for creating the object.
- `__init__` is responsible for initializing the object with any given values (like value in this case).

# The Python `__init__` Method

- The `__init__` method is similar to constructors in C++ and Java. It is run as soon as an object of a class is instantiated. The method is useful to do any initialization you want to do with your object.
- After `__new__` has created the instance, Python automatically calls `__init__` to initialize the instance with the provided arguments.
- `__init__` doesn't return anything; its return value is implicitly `None`.

# Creating a class and object with class and instance attributes

```
 class Dog:
         # class attribute
    attr1 = "mammal"

         # Instance attribute
    def __init__(self, name):
        self.name = name

# Driver code
# Object instantiation
Rodger = Dog("Rodger")
Tommy = Dog("Tommy")

# Accessing class attributes
print("Rodger is a {}".format(Rodger.__class__.attr1))
print("Tommy is also a {}".format(Tommy.__class__.attr1))

# Accessing instance attributes
print("My name is {}".format(Rodger.name))
print("My name is {}".format(Tommy.name))
```

## **\_\_str\_\_() method**

- Python has a particular method called `__str__()`, that is used to define how a class object should be represented as a string. It is often used to give an object a human-readable textual representation, which is helpful for logging, debugging, or showing users object information.
- When a class object is used to create a string using the built-in functions `print()` and `str()`, the `__str__()` function is automatically used. You can alter how objects of a class are represented in strings by defining the `__str__()` method.

## Python3



```
class GFG:
```

```
    def __init__(self, name, company):
```

```
        self.name = name
```

```
        self.company = company
```

```
    def __str__(self):
```

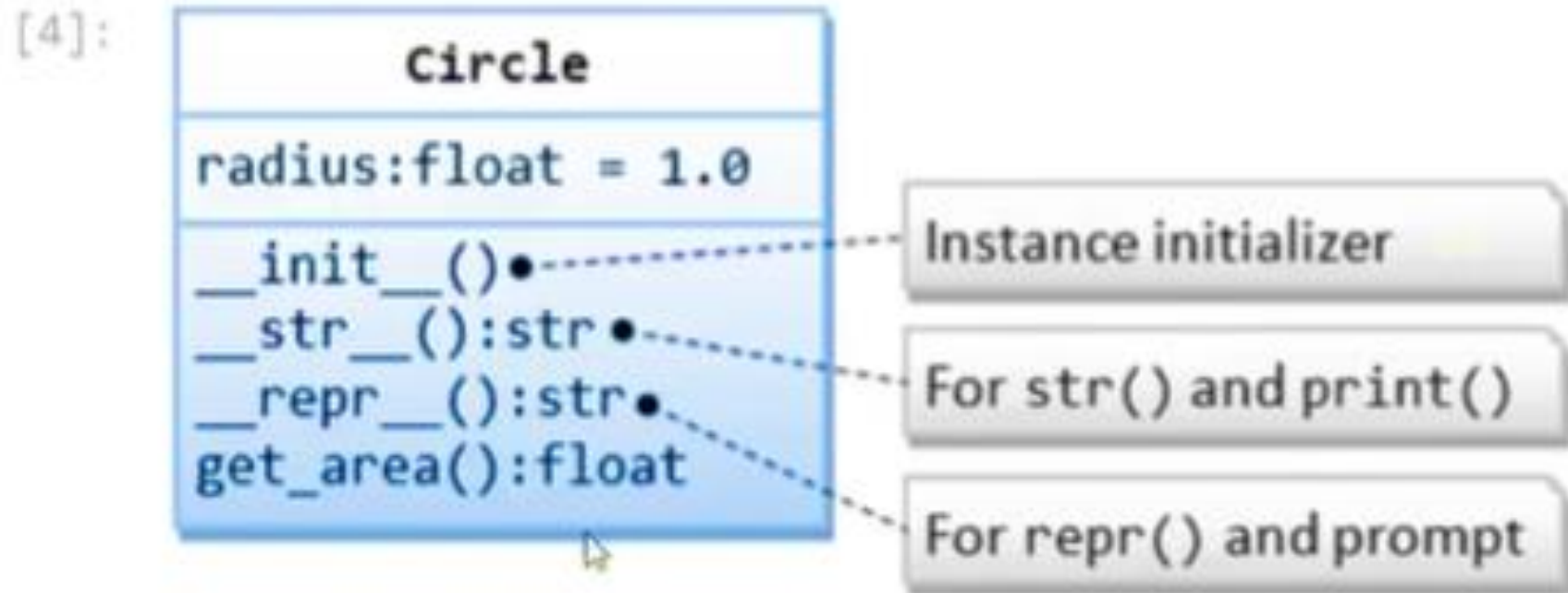
```
        return f"My name is {self.name} and I work in {self.company}."
```

```
my_obj = GFG("John", "GeeksForGeeks")
```

```
print(my_obj)
```

## Example - using a toy circle class

```
[4]: Image(r'oop-01.PNG', width=400)
```



## \_\_repr\_\_() method

- The `__repr__` method is meant to provide a detailed and unambiguous string representation of an object that is useful for debugging and development. The goal is to provide information that can help you understand the object's state or even recreate it if needed, often including enough information to recreate the object if needed.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Person(name={self.name!r}, age={self.age!r})"

person = Person("Alice", 30)
print(repr(person))
```

- `__repr__` is implemented to return a string that includes the class name and the attributes of the object.
- `!r` inside the f-string calls the `repr()` of the `name` and `age` attributes, which ensures that the string representation of these values is used.

## initialize instance variables

```
[10]: class Circle:
      """A Circle instance models a circle with a radius"""

      def __init__(self, radius=1.0):
          """Initializer with default radius of 1.0"""
          self.radius = radius # Create an instance variable radius
```

- the `instance variables` are declared within the `init()` method
- `self` : The first parameter of all the member methods shall be an object called `self` (e.g., `init(self, ...)`), which binds to this instance (i.e., itself) during invocation.
- Every time a new `Circle` object is created, `.init()` sets the initial state of the object by assigning the values of the object's properties.
  - That is, `.init()` initializes each new instance of the class.
- We can give `.init()` any number of parameters, but the `first parameter` will always be a variable called `self`.
- When a `new class instance` is created, the instance is automatically passed to the `self` parameter in `.init()` so that `new attributes` can be defined on the object.
- Inside the `init()` method, the `self.radius = radius` defines an `instance variable radius`.

- the `instance variables` are declared within the `__init__()` method
- `self` : The first parameter of all the member methods shall be an object called `self` (e.g., `__init__(self, ...)`), which binds to this instance (i.e., itself) during invocation.
- Every time a new `Circle` object is created, `__init__()` sets the initial state of the object by assigning the values of the object's properties.
  - That is, `__init__()` initializes each new instance of the class.
- We can give `__init__()` any number of parameters, but the `first parameter` will always be a variable called `self`.
- When a `new class instance` is created, the instance is automatically passed to the `self` parameter in `__init__()` so that `new attributes` can be defined on the object.
- Inside the `__init__()` method, the `self.radius = radius` defines an `instance variable` `radius`.
- Take note that:
  - `__init__()` is `not really the constructor`, but an `initializer` to create the instance variables.
  - `__init__()` shall `never return a value`.
  - `__init__()` is `optional` and can be omitted if there is no instance variables.

## Class / instance attributes

- Attributes created in `.init()` are called `instance attributes`.
  - An `instance attribute's` value is `specific` to a `particular instance` of the class.
  - All Dog objects have a name and an age, but the values for the name and age attributes will vary depending on the Dog instance.
- `class attributes` are attributes that have the same value for all `class instances`.
  - You can define a `class attribute` by assigning a value to a variable name `outside of .__init__()`.

```
] class Circle:
    """A Circle instance models a circle with a radius"""
    shape_object = 'Circle'

    def __init__(self, radius=1.0):
        """Initializer with default radius of 1.0"""
        self.radius = radius # Create an instance variable radius
```

# Destructors

- Destructors are called when an object gets destroyed. In Python, destructors are not needed as much as in C++ because Python has a garbage collector that handles memory management automatically.
- The `__del__()` method is known as a destructor method in Python. It is called when all references to the object have been deleted i.e when an object is garbage collected.

# Destructors

- Python employs a garbage collection mechanism that automatically handles the cleanup of resources and memory when objects are no longer in use. This is typically done through a process called reference counting and a cycle detector that identifies and collects objects with circular references.
- Python does provide a special method, `__del__`, which can be used as a kind of "destructor." This method is automatically called just before an object is about to be destroyed. It is important to note that using `__del__` is not always recommended, as relying on it can lead to unpredictable behavior, especially in the presence of circular references.

## Python3

```

# Python program to illustrate destructor
class Employee:

    # Initializing
    def __init__(self):
        print('Employee created.')

    # Deleting (Calling destructor)
    def __del__(self):
        print('Destructor called, Employee deleted.')

obj = Employee()
del obj

```

## Python3

```

# Python program to illustrate destructor

class Employee:

    # Initializing
    def __init__(self):
        print('Employee created')

    # Calling destructor
    def __del__(self):
        print("Destructor called")

def Create_obj():
    print('Making Object...')
    obj = Employee()
    print('function end...')
    return obj

print('Calling Create_obj() function...')
obj = Create_obj()
print('Program End...')

```

## Python

```

class RecursiveFunction:
    def __init__(self, n):
        self.n = n
        print("Recursive function initialized with n =", n)

    def run(self, n=None):
        if n is None:
            n = self.n
        if n <= 0:
            return
        print("Running recursive function with n =", n)
        self.run(n-1)

    def __del__(self):
        print("Recursive function object destroyed")

# Create an object of the class
obj = RecursiveFunction(5)

# Call the recursive function
obj.run()

# Destroy the object
del obj

```

```
class MyClass:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} created")

    def __del__(self):
        print(f"{self.name} is about to be destroyed")

# Creating instances of MyClass
obj1 = MyClass("Object 1")
obj2 = MyClass("Object 2")

# Deleting references to the objects (not recommended in practice)
del obj1
del obj2
```

- In this example, the `__del__` method is called just before an object is destroyed. However, relying on `__del__` for resource cleanup is not the preferred practice in Python. Instead, it's recommended to use context managers (with statement) for resource management, and let Python's garbage collector take care of memory management.

```
class Resource:
    def __enter__(self):
        print("Resource acquired")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Resource released")

# Using a context manager for resource management
with Resource() as res:
    # Code that uses the resource
    print("Doing something with the resource")
```

# Context Managers and the **with** Statement

- When you use the with statement, you're working with context managers. These context managers handle the resource management for you. Context managers can be used for various purposes, such as managing files, locks, network connections, and more.

# How the with Statement Works

The `with` statement takes an object instance that implements the context manager interface as an argument. Inside the `with` block, the code is executed at the time the block is entered. After the block, the code is executed, ensuring proper cleanup and teardown of the resource.

```
# Example: Reading from a file using a context manager
with open("example.txt", "r") as file:
    content = file.read()
# 'content' now contains the file contents
```

# Sub-objects

- Sub-objects, sometimes referred to as nested objects or object composition, are a means of "nesting" together or combining basic items to create more complex ones. Sub-objects are generated in object-oriented programming by defining one class inside the definition of another class. One or more instances of the inner class are contained in the outer class as data members, accessible and manipulatable just like any other data members.
- They enable objects to be divided into smaller, more manageable pieces, sub-objects can be a potent tool for creating complex object structures. This can facilitate reuse of current objects in different settings and make code easier to comprehend and maintain

```

class Engine:
    def __init__(self, fuel_type):
        self.fuel_type = fuel_type

    def start(self):
        print("Engine started")

    def stop(self):
        print("Engine stopped")

class Car:
    def __init__(self, make, model, engine):
        self.make = make
        self.model = model
        self.engine = engine

    def start(self):
        print(f"{self.make} {self.model}'s engine is starting.")
        self.engine.start()

    def stop(self):
        print(f"{self.make} {self.model}'s engine is stopping.")
        self.engine.stop()

# Creating an Engine instance
car_engine = Engine(fuel_type="Gasoline")

# Creating a Car instance with the Engine as a sub-object
my_car = Car(make="Toyota", model="Camry", engine=car_engine)

# Using methods of the Car and Engine classes
my_car.start()
my_car.stop()

```